

宁波大学
NINGBO UNIVERSITY

宁波大学信息学院

实验报告

实验课程名称： 嵌入式系统

学　　院： 信息科学与工程学院

专　　业： 计算机科学与技术

指 导 教 师： 王翀,王艺睿

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 2022 级计算机 2 班

学　　期： 2024-2025 学年第二学期

信息学院实验中心制

实验								
成绩								

实验项目名称	实验二——交叉开发环境搭建

一、实验目的与要求：

1.1 实验目的

(1) 熟悉嵌入式 Linux 交叉开发环境的搭建与使用。

(2) 掌握在宿主机上进行软件编辑、编译与连接，以及将编译好的程序下载到目标机运行调试的过程。

1.2 实验要求

(1) 配置虚拟机网络环境，确保宿主机与目标机可以进行网络通信。

(2) 配置交叉工具链，设置环境变量以便于使用交叉编译工具。

(3) 准备实验环境，包括拷贝必要的文件到指定目录，以及设置串口调试工具等。

二、实验设备：

(1) 华清远见开发环境。

(2) 实验平台为华清远见提供的虚拟机环境。

(3) 使用的操作系统是 Ubuntu 12.04 LTS 64-bit。

(4) 虚拟机工具软件为 VMware Player。

(5) FS4412 开发板。

三、实验方法（原理、流程图）

实验原理：

本实验的目的是利用 TFTP（Trivial File Transfer Protocol） 下载 Linux 内核，并使其在开发板上运行，同时使用 NFS（Network File System） 挂载根文件系统，为后续的开发提供便捷的调试环境。

1. TFTP 方式下载内核

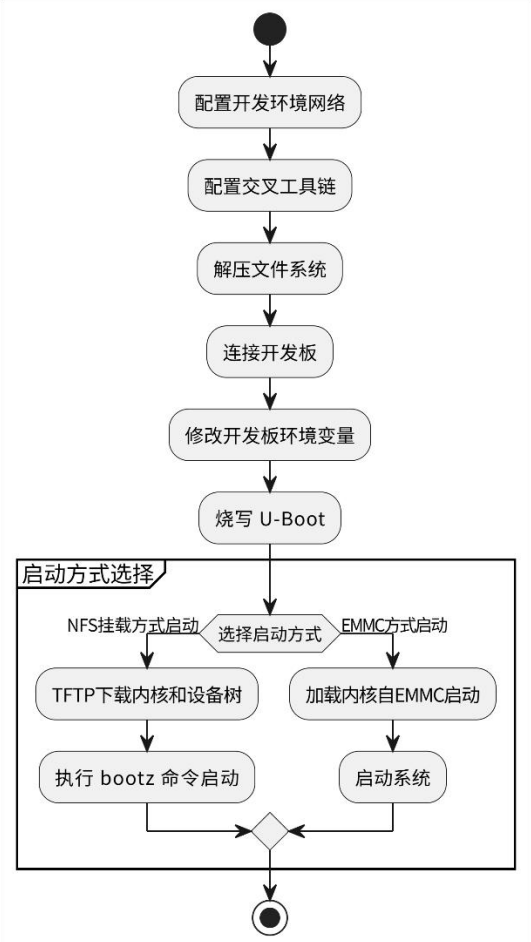
TFTP 是一种简化的文件传输协议，通常用于无盘工作站或嵌入式设备启动 Linux 内核。本实验中，TFTP 服务器运行在主机端，开发板通过 U-Boot 加载内核镜像 (zImage 或 uImage) 并启动 Linux 内核。

2. NFS 挂载根文件系统

NFS 是一种基于网络的文件系统，它允许不同设备共享文件。实验中，根文件系统存放在主机的 NFS 服务器上，开发板通过网络挂载该文件系统，使其作为系统的根目录，从而实现文件共享、调试和动态更新。

通过这种方式，避免了每次修改根文件系统都需要重新烧录 Flash，提高了开发效率。

流程图：



四、实验过程、步骤及内容

4.1 配置开发环境网络

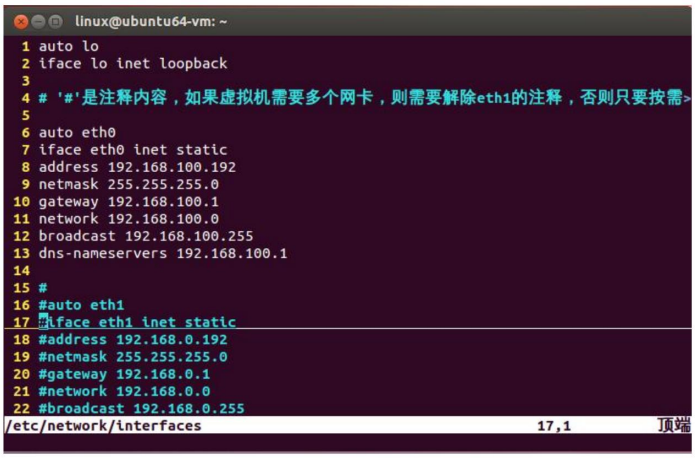
虚拟机网络方式为桥接模式，此状态下虚拟机下的操作系统和主机操作系统为平级状态。为了调试方便，我们可以给虚拟机下的 Ubuntu 一个静态的 IP 地址。

假设我们使用的网络地址为 192.168.100.x 段的，那么我可以给 Ubuntu 分配一个 IP 为 192.168.100.192，配置过程如下所示。

配置虚拟机网络环境。

```
$ sudo    vim    /etc/network/interfaces
```

修改文件如下图所示。保存退出（环境默认是加【#】注释掉的内容，修改前删除 4-11 行每行的注释）。

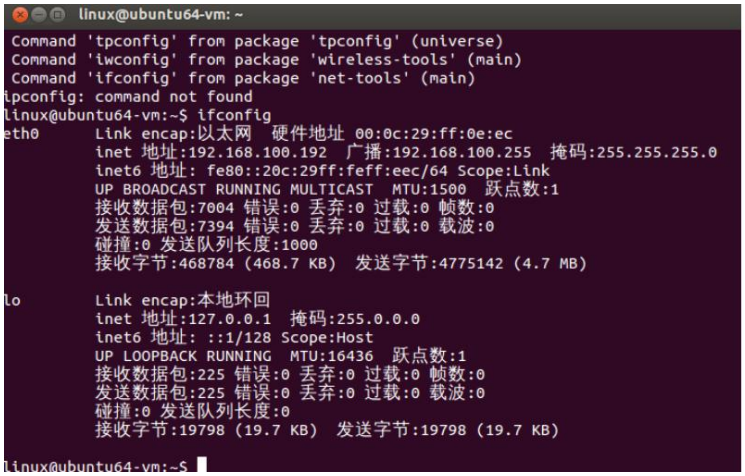


应用网络修改。

```
$ sudo    /etc/init.d/networking    restart
```

```
linux@ubuntu64-vm:~$ sudo vim /etc/network/interfaces
linux@ubuntu64-vm:~$ sudo /etc/init.d/networking restart
 * Running /etc/init.d/networking restart is deprecated because it may not enabl
e again some interfaces
 * Reconfiguring network interfaces...
ssh stop/waiting
ssh start/running, process 3205
                                                                        [ OK ]
linux@ubuntu64-vm:~$
```

如上图所示表明 IP 修改成功。使用【ifconfig】命令查看我们修改的结果。如果没有修改成功，重复上述步骤，或者重新启动虚拟机 Ubuntu 系统即可。



4.2 配置交叉工具链

华清远见开发环境包含了 3 个版本的交叉工具链，路径在/usr/local/toolchain/下，不必再次解压，直接指向路径即可。

一般情况下，当我们输入一个 Linux 命令，比如【ls】可以直接执行功能，但我当前的目录并没有【ls】这个文件。【ls】这个命令文件在【/bin】这个目录中，我们可以执行这是因为这些命令所在的路径包含在了用户的环境变量中。我们为了使用方便也将我们经常使用的交叉工具链添加到环境变量中。

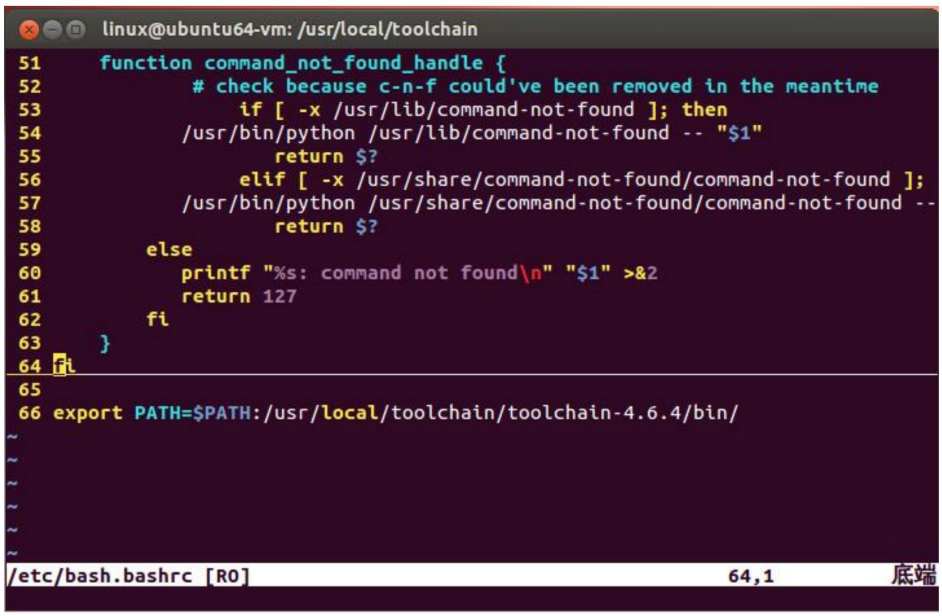
```
linux@ubuntu64-vm:~$ cd /usr/local/toolchain/
linux@ubuntu64-vm:/usr/local/toolchain$ ls
toolchain-4.3.2  toolchain-4.4.6  toolchain-4.5.1
linux@ubuntu64-vm:/usr/local/toolchain$
```

修改文件~/.bashrc，添加如下内容

```
$ vim    /etc/bash.bashrc        // 注意 bashrc 前面有句点
```

添加下面一行代码到文件的末尾。

```
export    PATH=$PATH:/usr/local/toolchain/toolchain-4.4.6/bin/
```



重启配置文件和工具链的测试

```
$ source    /etc/bash.bashrc
$ arm-none-linux-gnueabi-gcc    –v
```

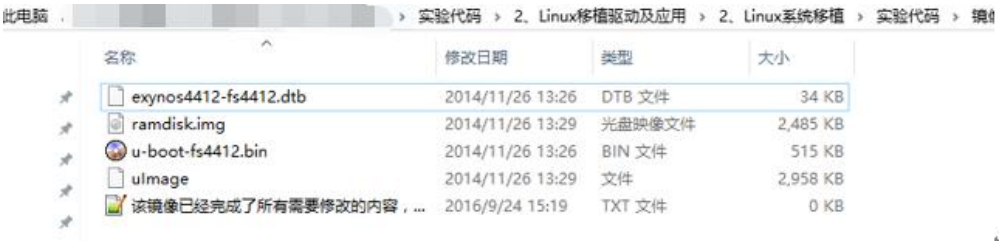
```
linux@ubuntu64-vm:/usr/local/toolchain$ source /etc/bash.bashrc
linux@ubuntu64-vm:/usr/local/toolchain$ arm-none-linux-gnueabi-gcc -v
Using built-in specs.
COLLECT_GCC=arm-none-linux-gnueabi-gcc
COLLECT_LTO_WRAPPER=/usr/local/toolchain/toolchain-4.6.4/bin/../libexec/gcc/arm-
arm1176jzfssf-linux-gnueabi/4.6.4/lto-wrapper
Target: arm-arm1176jzfssf-linux-gnueabi
Configured with: /work/builddir/src/gcc-4.6.4/configure --build=i686-build_pc-li
nux-gnu --host=i686-build_pc-linux-gnu --target=arm-arm1176jzfssf-linux-gnueabi
--prefix=/opt/TuxamitoSoftToolchains/arm-arm1176jzfssf-linux-gnueabi/gcc-4.6.4 -
-with-sysroot=/opt/TuxamitoSoftToolchains/arm-arm1176jzfssf-linux-gnueabi/gcc-4.
6.4/arm-arm1176jzfssf-linux-gnueabi/sysroot --enable-languages=c,c++ --with-arch
=armv6zk --with-cpu=arm1176jzf-s --with-tune=arm1176jzf-s --with-fpu=vfp --with-
float=softfp --with-pkgversion='crosstool-NG hg+default-2685dfa9de14 - tc0002' -
-disable-sjlj-exceptions --enable-__cxa_atexit --disable-libmudflap --disable-li
bgomp --disable-libssp --disable-libquadmath --disable-libquadmath-support --wit
h-gmp=/work/builddir/arm-arm1176jzfssf-linux-gnueabi/buildtools --with-mpfr=/wor
k/builddir/arm-arm1176jzfssf-linux-gnueabi/buildtools --with-mpc=/work/builddir/
arm-arm1176jzfssf-linux-gnueabi/buildtools --with-ppl=/work/builddir/arm-arm1176
jzfssf-linux-gnueabi/buildtools --with-cloog=/work/builddir/arm-arm1176jzfssf-li
nux-gnueabi/buildtools --with-libelf=/work/builddir/arm-arm1176jzfssf-linux-gnue
abi/buildtools --with-host-libstdcxx='-static-libgcc -Wl,-Bstatic,-lstdc++,-Bdyn
amic -lm' --enable-threads=posix --enable-target-optspace --without-long-double-
128 --disable-nls --disable-multilib --with-local-prefix=/opt/TuxamitoSoftToolch
ains/arm-arm1176jzfssf-linux-gnueabi/gcc-4.6.4/arm-arm1176jzfssf-linux-gnueabi/s
ysroot --enable-c99 --enable-long-long
Thread model: posix
gcc version 4.6.4 (crosstool-NG hg+default-2685dfa9de14 - tc0002)
linux@ubuntu64-vm:/usr/local/toolchain$
```

直接输入命令，即可执行，这样说明我们的交叉工具链就安装好了

4.3 拷贝文件

拷贝华清远见-CORTEXA9 资料：\实验代码\2、Linux 移植驱动及应用\2、Linux 系统移植\实验代码\镜像中的所有文件拷贝到 Ubuntu 共享目录 D:\share 下（1.3.3 节设置的位置）。通过共享目录

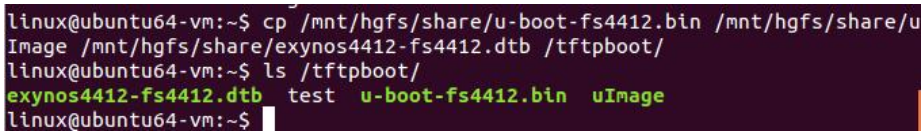




将共享目录中需要下载的文件拷贝到 tftp 目录中

拷贝 u-boot-fs4412.bin、uImage、exynos4412-fs4412.dtb 文件到虚拟机 Ubuntu 下的/tftpboot 目录下。

```
$ cp /mnt/hgfs/share/u-boot-fs4412.bin /mnt/hgfs/share/uImage /mnt/hgfs/share/exynos4412-fs4412.dtb /tftpboot/
```



4.4 解压文件系统

拷贝华清远见-CORTEXA9 资料：\实验代码\2、Linux 移植驱动及应用\2、Linux 系统移植\实验代码\移植后的源码\rootfs.tar.xz 文件到虚拟机 Ubuntu 下的/source 目录下。

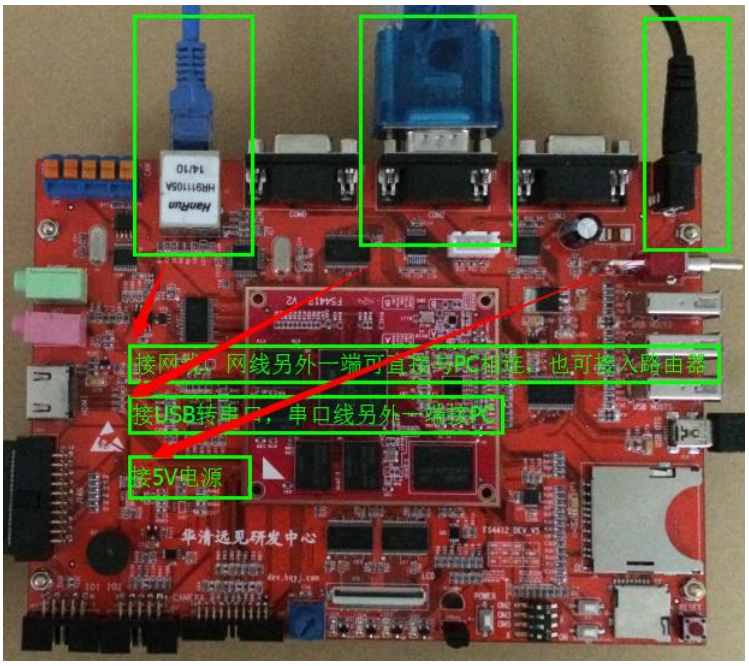
```
$  cp  /mnt/hgfs/share/rootfs.tar.xz  /source/
$  cd /source
$  tar  xvf  rootfs.tar.xz
```

```
linux@ubuntu64-vm:~$ cp /mnt/hgfs/share/rootfs.tar.xz /source/
linux@ubuntu64-vm:~$ ls /source/
rootfs  rootfs.tar.xz
```

```
rootfs/bin/busybox
rootfs/bin/kill
rootfs/bin/cp
rootfs/bin/gunzip
rootfs/bin/df
rootfs/bin/sleep
rootfs/bin/pwd
rootfs/bin/adduser
rootfs/bin/cttyhack
rootfs/bin/getopt
rootfs/bin/sed
rootfs/bin/delgroup
rootfs/bin/ionice
rootfs/bin/chgrp
rootfs/bin/chown
rootfs/bin/watch
rootfs/bin/ping6
rootfs/bin/sync
rootfs/bin/mt
rootfs/bin/run-parts
rootfs/bin/rpm
rootfs/bin/su
rootfs/bin/mount
rootfs/bin/catv
rootfs/bin/tar
rootfs/proc/
rootfs/sys/
linux@ubuntu64-vm:/source$
```

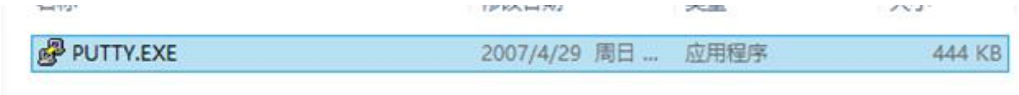
4.5 连接开发板

通过串口或其他方式连接开发板，准备进行开发板的启动和调试。设置串口调试工具，配置串口调试工具 (PuTTY) ，设置正确的端口和波特率， 以便与开发板通信。

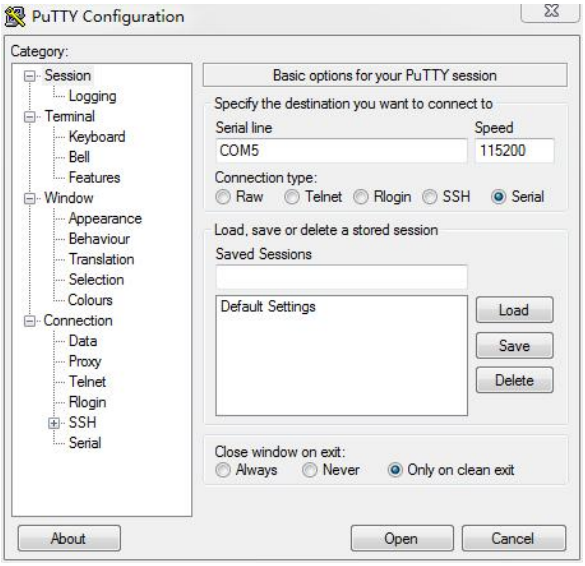


设置串口调试工具

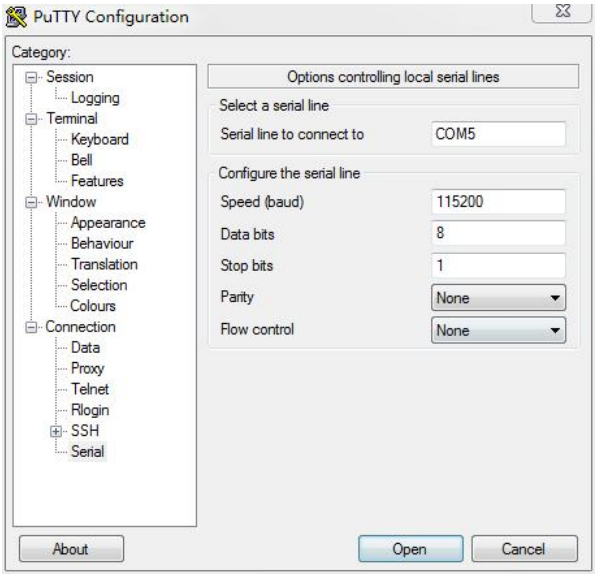
打开【华清远见-CORTEXA9 资料\工具软件\Windows\串口调试工具\putty.exe】文件。



选择串口（Serial）连接方式：



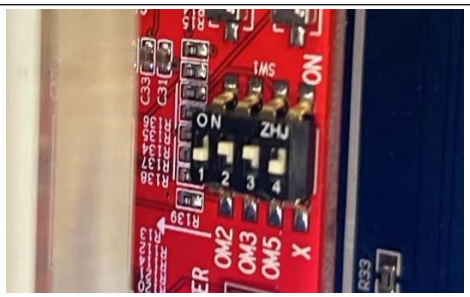
选中第一步方框内的 Serial,再点击第二步中的 Serial,进入串口设置的对话框：



点击 open 打开串口。

4.6 启动开发板

根据开发板的启动方式，设置拨码开关 (0110) ，然后启动开发板，如下图所示。



启动开发板，在倒计时结束前，按任意键停止在 Uboot 处，串口终端显示图下图所示。

```
T

U-Boot 2013.01 (Nov 10 2016 - 11:50:17) for FS4412

CPU:    Exynos4412@1000MHz

Board: FS4412
DRAM:  1 GiB
WARNING: Caches not enabled
PMIC: S5M8767(VER5.0)
MMC:   MMC0:    14910 MB
In:    serial
Out:   serial
Err:   serial

MMC read: dev # 0, block # 48, count 16 ...16 blocks read: OK
eMMC CLOSE Success.!!


Checking Boot Mode ... EMMC4.41
Net:   dm9000
Hit any key to stop autoboot:  0
FS4412 #
```

修改开发板环境变量（命令框内开头为【#】一般是需要在串口终端对开发进行的操作，【$】一般是在虚拟机下对 Ubuntu 进行的操作，下面省略此说明）。尽量手动输入，不去复制 pdf，避免非法字符。

```
# setenv serverip 192.168.100.192   //主机的 IP 地址，和 3.4.1 节，设置的 Ubuntu IP 地址一致
# setenv ipaddr 192.168.100.191     //板子的 IP，不要和 Windows 或 Ubuntu 冲突
# saveenv                           //保存环境变量
```

```
FS4412 # setenv serverip 192.168.100.192
FS4412 # setenv ipaddr 192.168.100.191
FS4412 # saveenv
Saving Environment to MMC...
Writing to MMC(0)... .done
```

使用【print】命令查看修改后的环境变量。

```
Unknown command 'printf' - try 'help'
FS4412 # print
baudrate=115200
bootargs=root=/dev/nfs nfsroot=192.168.100.192:/source/rootfs rw ip=192.168.100.
191 init=/linuxrc console=ttySAC2,115200
bootcmdbf=tftp 41000000 uImage; tftp 42000000 exynos4412-fs4412.dtb; bootm 41000
000 - 42000000
bootdelay=3
ethact=dm9000
ethaddr=11:22:33:44:55:66
fileaddr=41000000
filesize=26D213
gatewayip=192.168.100.1
ipaddr=192.168.100.191
ippr=192.168.100.191
netmask=255.255.255.0
serverip=192.168.100.192
setrip=192.168.100.192
stderr=serial
stdin=serial
stdout=serial

Environment size: 533/16380 bytes
FS4412 #
```

使用 ping 命令尝试 ping 一下 Ubuntu 主机，如下图所示，表示网络已经联通。

```
# ping  192.168.100.192
```

```
FS4412 # ping 192.168.100.192
dm9000 i/o: 0x5000000, id: 0x90000a46
DM9000: running in 16 bit mode
MAC: 11:22:33:44:55:66
operating at 100M full duplex mode
Using dm9000 device
host 192.168.100.192 is alive
FS4412 #
```

4.7 烧写 uboot

使用 tftp 命令下载 uboot 到开发板，并使用 movi 命令烧写 uboot 到 EMMC 或其他存储 介质。

在 uboot 命令行下，执行命令：

```
#  tftp  40008000  u-boot-fs4412.bin
#  movi  write  u-boot  40008000
```

```
FS4412 # tftp 40008000 u-boot-fs4412.bin
dm9000 i/o: 0x5000000, id: 0x90000a46
DM9000: running in 16 bit mode
MAC: 11:22:33:44:55:66
operating at 100M full duplex mode
Using dm9000 device
TFTP from server 192.168.100.192; our IP address is 192.168.100.191
Filename 'u-boot-fs4412.bin'.
Load address: 0x40008000
Loading: T ################################################################
         ##################################
         86.9 KiB/s
done
Bytes transferred = 527104 (80b00 hex)
FS4412 #
```

重启开发板，如下图所示。

```
T

U-Boot 2013.01 (Nov 10 2016 - 11:50:17) for FS4412

CPU:    Exynos4412@1000MHz

Board: FS4412
DRAM:  1 GiB
WARNING: Caches not enabled
PMIC: S5M8767(VER5.0)
MMC:   MMC0:    14910 MB
In:    serial
Out:   serial
Err:   serial

MMC read: dev # 0, block # 48, count 16 ...16 blocks read: OK
eMMC CLOSE Success.!!


Checking Boot Mode ... EMMC4.41
Net:   dm9000
Hit any key to stop autoboot:  0
FS4412 #
```

4.8 NFS 挂载方式启动

修改开发板环境变量。

```
# setenv serverip 192.168.100.192
# setenv ipaddr 192.168.100.191
# setenv gatewayip 192.168.100.1
# setenv bootcmd tftp 41000000 uImage\; tftp 42000000 exynos4412-fs4412.dtb \; bootm 41000000 - 42000000 # setenv bootargs root=/dev/nfs nfsroot=192.168.100.192:/source/rootfs rw ip=192.168.100.191 init=/linuxrc console=ttySAC2,115200 # saveenv
```

重启开发板，如下图启动，表明内核从主机的/tftpboot 处下载，文件系统为 nfs 网络文件系统，位置为主机的/source/rootfs/处。

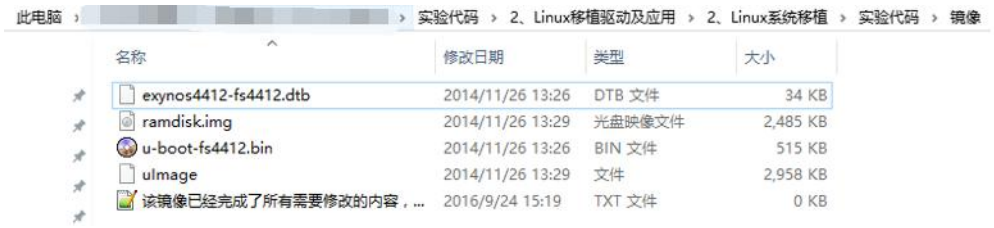
```
[    1.535000] mmcblk0rpmb: mmc1:0001 AGND3R partition 3 4.00 MiB
[    1.580000] hub 1-0:1.0: 3 ports detected
[    1.580000]  mmcblk0: p1 p2 p3 p4
[    1.590000] drivers/rtc/hctosys.c: unable to open rtc device (rtc0)
[    1.590000]  mmcblk0boot1: unknown partition table
[    1.595000]  mmcblk0boot0: unknown partition table
[    1.625000] dm9000 5000000.ethernet eth0: link down
[    1.765000] mmc0: new high speed SDHC card at address aaaa
[    1.770000] mmcblk1: mmc0:aaaa SS08G 7.40 GiB
[    1.785000]  mmcblk1: p1
[    1.850000] IP-Config: Guessing netmask 255.255.255.0
[    1.850000] IP-Config: Complete:
[    1.855000] dm9000 5000000.ethernet eth0: link up, 100Mbps, full-duplex, lpa
[    1.860000]      device=eth0, hwaddr=00:0a:2d:a6:55:a2, ipaddr=192.168.100.19
[    1.875000]      host=192.168.100.191, domain=, nis-domain=(none)
[    1.880000]      bootserver=255.255.255.255, rootserver=192.168.100.192, root
[    1.885000] clk: Not disabling unused clocks
[    1.900000] usb 1-3: new high-speed USB device number 2 using exynos-ehci
[    2.035000] hub 1-3:1.0: USB hub found
[    2.035000] hub 1-3:1.0: 3 ports detected
[    2.975000] VFS: Mounted root (nfs filesystem) on device 0:10.
[    2.980000] devtmpfs: mounted
[    2.980000] Freeing unused kernel memory: 228K (c0530000 - c0569000)
```

EMMC 方式启动

将【华清远见-CORTEXA9 资料:\实验代码\2、Linux 移植驱动及应用\2、Linux 系统移

植\实验代码\镜像】下的 U-boot（u-boot-fs4412.bin）、内核镜像（uImage）、设备树（exynos4412-fs4412.dtb）、文件系统（ramdisk.img）拷贝到虚拟机/tftpboot 目录下：

若用户已经拷贝这些文件到虚拟机/tftpboot 目录下，则不需要再次拷贝。





a) 烧写 U-boot 到 EMMC 上

```
# tftp 40008000 u-boot-fs4412.bin
# movi write u-boot 40008000
```

```
Checking Boot Mode ... EMMC4.41
Net:   dm9000
Hit any key to stop autoboot:  0
FS4412 # tftp 40008000 u-boot-fs4412.bin
dm9000 i/o: 0x5000000, id: 0x90000a46
DM9000: running in 16 bit mode
MAC: 11:22:33:44:55:66
operating at 10M full duplex mode
Using dm9000 device
TFTP from server 192.168.100.192; our IP address is 192.168.100.191
Filename 'u-boot-fs4412.bin'.
Load address: 0x40008000
Loading: T #################################################################

#####################################

78.1 KiB/s
done
Bytes transferred = 527104 (80b00 hex)
FS4412 # movi write u-boot 40008000
writing bootloader.. 0, 1038

MMC write: dev # 0, block # 0, count 1038. 1038 blocks write finish
1038 blocks verify1: OK
eMMC CLOSE Success.!!
completed
FS4412 #
```

b) 烧写内核镜像到 EMMC 上

```
# tftp 41000000 uImage
# movi write kernel 41000000
```

```
FS4412 # tftp 41000000 uImage
dm9000 i/o: 0x5000000, id: 0x90000a46
DM9000: running in 16 bit mode
MAC: 11:22:33:44:55:66
operating at 10M full duplex mode
Using dm9000 device
TFTP from server 192.168.100.192; our IP address is 192.168.100.191
Filename 'uImage'.
Load address: 0x41000000
Loading: T #################################################################

#################################################################

#################################################################

#################################################################

#################################################################

#################################################################

#################################################################
```

```
249 KiB/s
done
Bytes transferred = 3028040 (2e3448 hex)
FS4412 # movi write kernel 41000000
writing kernel.. 1120, 8192
MMC write: dev # 0, block # 1120, count 8192. 8192 blocks write fin
8192 blocks written: OK
completed
FS4412 #
```

c) 烧写设备树文件到 EMMC 上

```
# tftp 41000000 exynos4412-fs4412.dtb
# movi write dtb 41000000
```

```
FS4412 # tftp 41000000 exynos4412-fs4412.dtb
dm9000 i/o: 0x5000000, id: 0x90000a46
DM9000: running in 16 bit mode
MAC: 11:22:33:44:55:66
operating at 10M full duplex mode
Using dm9000 device
TFTP from server 192.168.100.192; our IP address is 192.168.100.191
Filename 'exynos4412-fs4412.dtb'.
Load address: 0x41000000
Loading: T #######
         5.9 KiB/s
done
Bytes transferred = 34358 (8636 hex)
FS4412 # movi write dtb 41000000
writing devicetree.. 9312, 2048
MMC write: dev # 0, block # 9312, count 2048. 2048 blocks write finish
2048 blocks verifyl: OK
completed
FS4412 #
```

d) 烧写文件系统镜像到 EMMC 上

```
# tftp 41000000 ramdisk.img
# movi write rootfs 41000000 300000（烧写大小）
```

```
FS4412 # tftp 41000000 ramdisk.img
dm9000 i/o: 0x5000000, id: 0x90000a46
DM9000: running in 16 bit mode
MAC: 11:22:33:44:55:66
operating at 10M full duplex mode
Using dm9000 device
TFTP from server 192.168.100.192; our IP address is 192.168.100.191
Filename 'ramdisk.img'.
Load address: 0x41000000
Loading: #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ########################################
         443.4 KiB/s
done
Bytes transferred = 2544147 (26d213 hex)
FS4412 # movi write rootfs 41000000 300000
writing RFS.. 11360, 6144
MMC write: dev # 0, block # 11360, count 6144. 6144 blocks write finish
6144 blocks written: OK
completed
FS4412 #
```

e) 设置启动参数(从 EMMC 模式自启动 完全脱离服务器)

#setenv bootarg

setenv bootcmd movi read kernel 41000000\;movi read dtb 42000000\;movi read rootfs 43000000 300000\;bootm 41000000 43000000 42000000

#setenv bootargs

saveenv

```
DM9000: running in 16 bit mode
MAC: 11:22:33:44:55:66
operating at 10M full duplex mode
Using dm9000 device
TFTP from server 192.168.100.192; our IP address is 192.168.100.191
Filename 'ramdisk.img'.
Load address: 0x41000000
Loading: #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ########################################
         443.4 KiB/s
done
Bytes transferred = 2544147 (26d213 hex)
FS4412 # movi write rootfs 41000000 300000
writing RFS.. 11360, 6144
MMC write: dev # 0, block # 11360, count 6144. 6144 blocks write finish
6144 blocks written: OK
completed
FS4412 # setenv bootcmd movi read kernel 41000000\;movi read dtb 42000000\;movi
read rootfs 43000000
FS4412 # 31000000 43000000 42000000
Unknown command '31000000' - try 'help'
FS4412 # setenv bootcmd movi read kernel 41000000\;movi read dtb 42000000\;movi
read rootfs 43000000 300000\;bootm 41000000 43000000 42000000
FS4412 # setenv bootargs
FS4412 # saveenv
Saving Environment to MMC...
Writing to MMC(0)... .done
FS4412 #
```

五、实验数据（现象）处理分析

在实验过程中，我遇到了开发板与 Ubuntu 主机无法互相 ping 通的问题，尝试了多种方法后未能解决，最终通过更换电脑和开发板后成功连接。

以下是关键实验现象及数据分析：

1. **网络连接配置：**
   - 配置 Ubuntu 的 IP 地址为 192.168.100.192，并设置虚拟机网络模式为桥接。
   - 通过 ifconfig 或 ip a 命令确认网络配置，确保开发板与主机在同一子网。
   - 在开发板 U-Boot 命令行中使用 ping 192.168.100.192 测试网络连通性。
   - 发现 ping 不通后，排查过程如下：
     - 确认 Ubuntu 防火墙状态，发现 ufw 默认是启用的，尝试关闭防火墙：sudo ufw disable。
     - 在 Ubuntu 上使用 tcpdump -i eth0 icmp 监听 ICMP 报文，确认开发板是否发送了 ping 请求。
     - 更换虚拟机的网络模式（桥接、NAT 等）进行测试，最终发现更换物理电脑后可以正常通信。
2. **TFTP 文件传输：**
   - 启动 TFTP 服务，并将 u-boot-fs4412.bin、uImage、exynos4412-fs4412.dtb 等文件放入/tftpboot 目录。
   - 确保 TFTP 服务器正在运行，并通过 netstat -ulnp | grep 69 确认 TFTP 端口开启。
   - 在 U-Boot 下执行 tftp 40008000 u-boot-fs4412.bin，成功下载镜像文件。
   - 传输后进行文件完整性校验，例如使用 md5sum 对比服务器端和开发板端的文件哈希值。
3. **NFS 挂载：**
   - 在 Ubuntu 上配置 NFS 服务器，确保/etc/exports 文件正确配置，例如：

```
/nfsroot *(rw,sync,no_root_squash,no_subtree_check)
```

   - 重新导出 NFS 目录：sudo exportfs -rv。
   - 在 U-Boot 环境变量中设置 nfsroot 路径，例如：

```
setenv bootargs root=/dev/nfs nfsroot=192.168.100.192:/nfsroot ip=dhcp
```

   - 执行 bootm 命令，成功启动系统并挂载根文件系统。
   - 挂载后测试读写权限，确保根文件系统可正常操作。
4. **EMMC 启动测试：**
   - 依次烧写 U-Boot、内核、设备树及根文件系统：

```
mmc write 0x40008000 0 1000   # 假设烧写 U-Boot
mmc write 0x42000000 1000 2000   # 假设烧写内核
```

   - 设置 bootcmd 和 bootargs，例如：

```
setenv bootcmd 'mmc read 0x42000000 1000 2000; bootm 0x42000000'
setenv bootargs 'console=ttySAC0,115200 root=/dev/mmcblk0p2 rw'
saveenv
```

   - 断电重启后，开发板能够独立从 EMMC 启动，验证成功。
   - 观察串口日志，确保内核正确加载，根文件系统成功挂载。

其余具体实验过程以及数据现象分析请见**四、实验过程、步骤及内容**

六、实验结论：

通过本次实验，我深入了解了嵌入式 Linux 交叉开发环境的搭建与使用。首先，我学会了使用 TFTP 传输文件到开发板上，这是在嵌入式系统中常见的一种文件传输方式，能够快速、高效地将固件和文件传输到目标设备上。其次，我通过配置网络环境，使得开发板与 Ubuntu 能够相互通信，这对于团队协作和远程调试至关重要。在此过程中，我遇到了网络连接问题，最终通过排查网络模式、监听 ICMP 报文等方式找到问题所在并解决。

在此基础上，我进行了烧写和 NFS 挂载的操作，烧写是将 U-Boot 和内核等关键文件写入到开发板的存储设备中，而 NFS 挂载则是将文件系统挂载到了 Ubuntu 的指定目录下，这样我可以方便地查看和操作开发板的文件系统，极大地提高了开发效率。

最后，我还实现了使用 EMMC 模式自启动，这是一种比较稳定可靠的启动方式，能够确保系统在断电后能够自动启动，而不需要手动干预。通过这次实验，我不仅掌握了具体的操作步骤，还深入理解了嵌入式系统启动的原理和流程。同时，我学会了在实验中遇到问题时，如何系统性地进行问题定位与解决，提升了实际动手能力和工程实践能力。

七、指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。